

CDAP/1.0

Code Duel Arena Protocol

Application-Layer Protocol Design

Computer Networks - Project 1: Socket Programming

สนามแข่งขันเขียนโปรแกรมแบบ real-time ที่ตรวจ correctness พร้อม empirical time/space complexity contract

Transport model: TCP authority + UDP display optimization
Implementation: Python 3.9+ standard library; optional Docker isolation

Protocol Specification and Threat Model

Bilingual Thai/English technical report

Contents / สารบัญ

- 1. Application objective and architecture
- 2. TCP/UDP transport-layer rationale
- 3. TCP framing, messages and state machine
- 4. Status namespaces and UDP datagrams
- 5. Complexity profiler and sandbox backends
- 6. Experimental results and demo matrix
- Appendix A. Threat model and accepted risks

Application-Layer Protocol Design for Socket Programming

เอกสารนี้เสนอ network application สำหรับวิชา Computer Networks ในหัวข้อ Socket Programming โดยออกแบบ protocol ระดับ application ชื่อ **CDAP (Code Duel Arena Protocol)** เวอร์ชัน **CDAP/1.0** พร้อมเหตุผลในการเลือก Transport Layer, รูปแบบ message, status code/status phrase, state machine และผลการทดสอบจาก implementation จริง

1. วัตถุประสงค์ของ Application

CDAP เป็นสนามแข่งขันเขียนโปรแกรมแบบ real-time ผู้เล่นสองคนได้รับโจทย์เดียวกันและส่ง Python source code ไปยัง Arena Server จากนั้น Judge Worker จะตรวจ correctness และวัด time/space complexity จากหลาย input sizes เพื่อตรวจตาม complexity contract ของโจทย์

จุดแตกต่างสำคัญคือ solution ที่ตอบถูกแต่ช้ากว่า contract จะไม่ผ่าน เช่น โจทย์กำหนด **required_time: 0(n)** แต่ระบบวัดได้ **0(n^2)** จะได้รับ verdict **606 TIME_COMPLEXITY_VIOLATION** การส่งข้อมูลสำเร็จจึงไม่เท่ากับ algorithm ผ่านการตัดสิน

ลักษณะของระบบ:

- Multi-client: รองรับผู้เล่นและ Judge Workers หลาย connection พร้อมกัน
- Real-time: มี matchmaking, match clock, progress และ leaderboard
- Asynchronous: **SUBMIT** ตอน **202 ACCEPTED** ก่อน แล้วส่ง **VERDICT** event ภายหลัง
- Auditable: verdict ส่งค่าที่วัดได้, fitted model, confidence และ backend กลับมาด้วย
- Fault-aware: ถ้า UDP หาย การแข่งขันยังถูกต้องผ่าน TCP; ถ้า Worker ตาย job ถูก requeue
- Explainable: wire format เป็น text และโปรแกรมพิมพ์ทุก message พร้อม status code/phrase

2. Architecture

```

Player A --TCP-->                                     <--TCP-- Judge Worker 1
Player B --TCP--> Arena Server + Queue                 <--TCP-- Judge Worker 2
Feed pane <--UDP--                                     <--TCP-- Judge Worker N
  
```

Arena Server เป็น authority ของ account, lobby, room, match, submission และ verdict ส่วน Worker เป็น client ที่เชื่อมต่อหา Arena แล้ว long-poll ด้วย **WORKER_PULL** จึงใช้งานหลัง NAT ได้โดยไม่ต้องเปิด inbound port ที่ Worker

3. การเลือก Transport Layer Service Model

CDAP ใช้ทั้ง TCP และ UDP เพราะข้อมูลสองกลุ่มมี requirement ต่างกัน

Traffic	Transport	เหตุผล
HELLO , auth, matchmaking, source code, verdict, match events, Worker jobs	TCP	ต้องครบทุก byte, ต้องรักษาลำดับ, ต้องตรวจ correlation และหายไม่ได้
TICK, CLOCK, BOARD live display	UDP	เป็น latest-value-wins; datagram ใหม่แทนค่าเก่าได้ทันที การ retransmit ค่าเก่าทำให้หน้าจอขยั่นหลัง

3.1 เหตุผลที่ TCP จำเป็น

Source code ที่ขาดหนึ่ง byte อาจกลายเป็น syntax error และลงโทษผู้เล่นผิดคน Verdict ต้องส่งถึง client และลำดับ event มีความหมาย เช่น **VERDICT** ต้องไม่ถูก frame อื่นแทรกกลาง TCP ให้ reliable ordered byte stream และ connection state ที่เหมาะกับข้อมูลเหล่านี้

ข้อควรระวังคือ TCP ไม่มี message boundary ดังนั้น CDAP ต้องสร้าง framing เองด้วย **Content-Length** และต้องรองรับหลาย frame ที่มาถึงใน **recv()** ครั้งเดียว

3.2 เหตุผลที่ UDP เหมาะกับ Live Feed

Progress และ clock ส่งประมาณ 4 ครั้ง/วินาที หาก packet หนึ่งหาย packet ถัดไปจะอัปเดต ค่าปัจจุบันในประมาณ 250 ms ไม่มี ACK และไม่มี retransmission ทุก datagram มี seq และ receiver ทิ้งค่าที่ $seq \leq last_seen$ จึงไม่แสดงข้อมูลย้อนเวลาเมื่อเกิด reordering

Design invariant: UDP เป็น optimization เท่านั้น ไม่มี state-changing operation, source code, secret หรือ verdict อยู่บนช่องนี้ การใช้ `--no-udp` ต้องยังเล่นและจบ match ได้

4. TCP Wire Format

```
CDAP/1.0 <START>\r\n
Header-Name: value\r\n
...\r\n
\r\n
<body exactly Content-Length bytes>
```

มี message สามชนิดและแยกได้จาก start line ดังนี้:

Kind	ตัวอย่าง	Detection
Request	CDAP/1.0 SUBMIT	token ที่สองเป็น method
Response	CDAP/1.0 202 ACCEPTED	token ที่สองเป็นตัวเลข
Event	CDAP/1.0 EVENT VERDICT	token ที่สองคือ EVENT

Request มี Seq เพิ่มขึ้นทีละหนึ่ง และ Response ต้อง echo Seq เดิม ส่วน Event มี Event-Id และ ไม่มี Seq ทำให้ reader thread route response ไปยัง caller ที่รออยู่ และ route event ไปยัง event handler โดยไม่ต้องเดา

4.1 ตัวอย่าง Submission

```
CDAP/1.0 SUBMIT
Seq: 7
Match: m-0001
Lang: python
Content-Length: 412
Body-SHA256: <64 hex characters>
```

```
def solve(nums):
    ...
```

Arena ตรวจสอบ Body-SHA256 ก่อนใช้ body หากไม่ตรงตอบ 422 BODY_HASH_MISMATCH

4.2 ตัวอย่าง Asynchronous Response และ Event

```
CDAP/1.0 202 ACCEPTED
Seq: 7
Submission: s-0001
Queue-Pos: 1
```

```
CDAP/1.0 EVENT VERDICT
Event-Id: 17
Submission: s-0001
Verdict: 606 TIME_COMPLEXITY_VIOLATION
Content-Type: application/json
Content-Length: 412
```

```
{"required_time": "O(n)", "inferred_time": "O(n^2)", "confidence": "high"}
```

202 หมายถึง Arena รับ job แล้ว ไม่ได้หมายถึง solution ผ่าน ส่วน 606 เป็น Judge Verdict ภายใน event ไม่ใช่ protocol response status

5. Method Catalogue

5.1 Session

Method	Success	Errors ที่สำคัญ
HELLO	200 OK + capability body	426 VERSION_UNSUPPORTED
REGISTER {user, pass}	201 REGISTERED	400 BAD_REQUEST, 409 USER_EXISTS
LOGIN {user, pass}	200 OK + token	401 AUTH_FAILED
LOGOUT	204 NO_CONTENT	401 AUTH_FAILED

Unknown user และ wrong password ใช้ 401 AUTH_FAILED เหมือนกันเพื่อไม่ให้ LOGIN เป็น username oracle

5.2 Lobby และ Private Room

Method	Success	Errors ที่สำคัญ
QUEUE	202 QUEUED	409 ALREADY_QUEUED
DEQUEUE	200 OK	409 NOT_QUEUED
CREATE_ROOM	201 CREATED	404 NOT_FOUND, 429 RATE_LIMITED
JOIN_ROOM	200 OK	404 ROOM_NOT_FOUND, 409 ROOM_FULL
READY	200 OK	403 NOT_IN_ROOM
LEAVE	204 NO_CONTENT	403 NOT_IN_ROOM
FORFEIT	200 OK	403 NOT_IN_MATCH

5.3 Problem และ Submission

Method	Success	Errors ที่สำคัญ
GET_PROBLEM	200 OK + statement/contract	403 NOT_IN_MATCH, 403 WRONG_STATE
SUBMIT	202 ACCEPTED	410 MATCH_ENDED, 413, 415, 422, 429, 503
GET_SUBMISSION	200 OK verdict หรือ 202 ACCEPTED stage	403 FORBIDDEN, 404 SUBMISSION_NOT_FOUND

ลำดับ validation ของ SUBMIT ถูกกำหนดโดยตั้งใจ: 415 UNSUPPORTED_LANGUAGE ตรวจสอบได้ก่อน match state และ 503 JUDGE_UNAVAILABLE จะไม่สร้าง submission record หรือเริ่ม cooldown

5.4 Remote Worker Pool

Method	ความหมาย
WORKER_REGISTER	ตรวจ pre-shared token และประกาศ backend/capability
WORKER_PULL	long-poll สูงสุด 25 s; 200 OK + job หรือ 204 NO_CONTENT
WORKER_HEARTBEAT	ต่ออายุ lease และรายงาน stage ที่สังเกตจริง
WORKER_RESULT	ส่ง verdict; result แรกชนะ result ซ้ำได้ 409 CONFLICT

หาก Worker connection หายหรือพลาด heartbeat สามครั้ง Arena eject Worker และ requeue job จึงเป็น at-least-once dispatch แต่ at-most-once authoritative verdict

6. Status Namespaces

CDAP แยก protocol status 1xx-5xx ออกจาก judge verdict 6xx

6.1 Protocol Status

Code	Phrase(s)
200	OK
201	CREATED, REGISTERED
202	ACCEPTED, QUEUED
204	NO_CONTENT
400	BAD_REQUEST
401	AUTH_FAILED
403	FORBIDDEN, NOT_IN_MATCH, NOT_IN_ROOM, WRONG_STATE
404	NOT_FOUND, ROOM_NOT_FOUND, SUBMISSION_NOT_FOUND
405	METHOD_NOT_ALLOWED
408	REQUEST_TIMEOUT
409	CONFLICT, ALREADY_QUEUED, NOT_QUEUED, ROOM_FULL, USER_EXISTS
410	MATCH_ENDED
413	PAYLOAD_TOO_LARGE
415	UNSUPPORTED_LANGUAGE
422	BODY_HASH_MISMATCH
426	VERSION_UNSUPPORTED
429	RATE_LIMITED, SUBMIT_COOLDOWN
500	INTERNAL_ERROR
503	JUDGE_UNAVAILABLE

6.2 Judge Verdict

Code	Phrase	ความหมาย
600	ACCEPTED	ถูกต้องและผ่าน complexity contract
601	WRONG_ANSWER	output value ผิด
602	TIME_LIMIT_EXCEEDED	parent kill เมื่อเกิน wall-clock
603	MEMORY_LIMIT_EXCEEDED	memory เกิน absolute cap
604	COMPILE_ERROR	parse/compile ไม่สำเร็จ
605	RUNTIME_ERROR	exception ระหว่างทำงาน
606	TIME_COMPLEXITY_VIOLATION	ถูกแต่ growth ช้ากว่า contract
607	SPACE_COMPLEXITY_VIOLATION	auxiliary-space growth เกิน contract
608	OUTPUT_FORMAT_ERROR	value เท่ากันแต่ type/shape ผิด
609	SANDBOX_VIOLATION	AST guard ปฏิเสธ operation
611	INDETERMINATE_COMPLEXITY	ไม่มี model ที่ fit อย่างน่าเชื่อถือ
612	JUDGE_ERROR	Judge ล้มเหลว ไม่ใช่ความผิดผู้เล่น

7. UDP Datagram Format

หนึ่ง datagram คือหนึ่ง message จึงต้องมี **Content-Length**

CDAP/1.0 ATTACH session=<token>

```
CDAP/1.0 CLOCK match=m-0001 seq=81 remain=42150
CDAP/1.0 TICK match=m-0001 seq=82 t=1724500000123 player=alice passed=7 total=10 subs=2
CDAP/1.0 BOARD match=m-0001 seq=83 e=alice:7:2,bob:10:1
```

Value ทุกตัว percent-encoded เพื่อไม่ให้ space หรือ = ทำลาย field boundary Server เรียนรู้ endpoint จาก source address ของ **ATTACH** ซึ่งเหมาะกับ NAT ไม่มี UDP ACK, retransmission หรือ state-changing request

8. Per-Connection State Machine

```
INIT --HELLO--> GREETED --LOGIN--> IDLE <--> QUEUED --MATCH_START--> IN_MATCH
                                     ^                               |
                                     +----- MATCH_END -----+
```

IDLE <--> IN_ROOM

Any state: protocol error -> response and remain; fatal framing error -> CLOSED

Request ผิด state ได้ 403 พร้อม phrase และ Detail ที่บอก current state ทำให้ state machine ทดสอบจาก wire ได้จริง

9. Complexity Measurement

Method A - Wall-Clock Regression

รันหก input sizes, warm-up หนึ่งครั้ง, วัดซ้ำห้าครั้งและใช้ค่าต่ำสุดเพื่อลด one-sided timing noise จาก scheduling ใช้ least squares through origin และ relative RMSE เปรียบเทียบ $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$

Method B - Opcode Counting

นับ Python bytecode instruction ด้วย `sys.monitoring` (PEP 669) และใช้หนึ่ง run ต่อ size เพราะ count deterministic Capability probe พบว่า `sys.settrace/f_trace_opcodes` บน CPython 3.14.3 เครื่องนี้คืน count เป็นศูนย์แบบ silent จึงห้ามใช้ mechanism ที่ไม่ผ่าน non-zero, reproducible และ scaling checks

Decision Policy

- `margin >= 1.15`: high confidence
- `margin < 1.15`: เลือก class ที่ถูกกว่าและระบุ low confidence
- best relative RMSE > 0.35: 611 INDETERMINATE_COMPLEXITY
- Method A เป็น authority; Method B เป็น second opinion และรายงาน disagreement

Policy เลือกเข้าข้างผู้เล่นเมื่อ ambiguous เพราะ false 606 ร้ายแรงกว่า borderline accept

10. Sandbox Backends

SubprocessBackend ใช้ fresh process, temporary cwd, stdin ที่ runner consume ก่อนโหลด code, 64 KB output cap, wall-clock tree kill และ sentinel result channel บน Windows memory limit เป็น best-effort เพราะไม่มี `setrlimit` หรือ cgroup

DockerBackend ใช้ image `python:3.14-slim`, `--network none`, read-only root filesystem, memory cgroup, `--pids-limit 8`, `--cap-drop ALL`, `no-new-privileges`, unprivileged user และ `/tmp tmpfs` ถ้า Docker daemon ใช้ไม่ได้ selector fallback เป็น subprocess และ verdict ต้องรายงาน `backend=subprocess` ตามที่รันจริง

AST guard เป็น defence-in-depth ไม่ใช่ security boundary และมี `--no-ast-guard` เพื่อพิสูจน์ข้อจำกัดนี้ใน experiment โดยไม่ปิด

11. Experimental Results (CPython 3.14.3, Windows 11)

ผลจาก `python -m experiments.confusion_matrix` จำนวน 8 solutions:

- Method A: 6/7 polynomial cases ตรงกับ known class (85.7%)

- Method B: 5/7 pure-Python cases ตรง (71.4%); 5/8 ทุก cases (62.5%)
- Methods disagree 1 case
- `has_duplicate_onlogn.py`: Method A= $O(n \log n)$, Method B= $O(n)$ ตาม C-builtin blind spot
- `has_duplicate_on.py`: run นี้ทั้ง Method A และ B ให้ indeterminate (611) แสดงว่า

empirical measurement ยังมี noise และไม่ควรวาง precision เกินข้อมูล

- `fib_naive.py`: model ไม่ fit แต่ raw slope backstop พิสูจน์ว่าเกิน $O(n)$ จึงได้ 606

ผล backend run ($n=3$ ต่อ backend; image ถูก provision ก่อนเริ่มจับเวลาส่งงาน):

- subprocess: mean 92.2 ms, median 92.6 ms, 10.847 submissions/s
- Docker: mean 2429.8 ms, median 1027.3 ms, 0.412 submissions/s

(รวม Windows container startup ต่อ submission; first warm run สูงกว่ารอบถัดไป)

- Guard off: socket, host-file และ 16-process probes สำเร็จได้ subprocess
- Guard off: ทั้งสามถูก block/confine ได้ Docker; fork probe สร้างได้ 7 ก่อนชน pid cap

12. Test and Demo Matrix

Scenario	Expected evidence
Good $O(n)$ solution	600 ACCEPTED
Correct $O(n^2)$ solution	606 TIME_COMPLEXITY_VIOLATION
<code>--bad-version</code>	426 VERSION_UNSUPPORTED
<code>--tamper</code>	422 BODY_HASH_MISMATCH
<code>--lang rust</code>	415 UNSUPPORTED_LANGUAGE
No healthy worker	503 JUDGE_UNAVAILABLE, no cooldown
Kill leased Worker	job requeued and replacement returns one verdict
<code>--udp-loss 0.4</code>	visible drops, later sequence converges
<code>--no-udp</code>	match and TCP verdict complete normally
<code>--backend docker</code>	verdict evidence says <code>backend=docker</code>
Guard off security probes	subprocess escape vs Docker block/confine

13. Conclusion

CDAP แสดง application-layer protocol ที่มี framing, request correlation, asynchronous events, state machine, status vocabulary, distributed workers และ transport split ที่มีเหตุผล TCP รับผิดชอบ correctness ส่วน UDP ปรับปรุง display โดยไม่กลายเป็น dependency ผลการทดลอง ยังแสดงข้อจำกัดของ empirical complexity inference และ sandbox อย่างตรงไปตรงมา

Appendix A - Threat Model / แบบจำลองภัยคุกคาม

1. Scope

เอกสารนี้อธิบาย security boundary ของ Arena, Judge Worker, submission sandbox, TCP control channel และ UDP live feed สำหรับ coursework deployment บนเครื่องเดียวหรือ LAN ที่เชื่อถือได้ ไม่อ้างว่า CDAP/1.0 พร้อมใช้งานบน public Internet

2. Assets

- ความถูกต้องของ match state, clock, submission ownership และ verdict
- Source code ของผู้เล่นก่อน match จบ
- Availability ของ Arena และ Judge Workers
- Host filesystem, process table, network access และ memory
- Worker pre-shared token และ player session/feed token
- ความจริงของ evidence เช่น **backend**, measured complexity และ status log

3. Adversaries

1. Malicious contestant ส่ง source code เพื่ออ่านไฟล์, เปิด network, spawn process หรือใช้ resource
2. Malformed client ส่ง frame ใหญ่, hash ผิด, method/state ผิด หรือ header injection
3. Fake Worker พยายาม pull source code หรือส่ง verdict ปลอม
4. UDP injector ส่ง display data ปลอม/reordered; ผลกระทบต้องจำกัดอยู่ที่หน้าจอ
5. Failed Worker หายกลาง job หรือส่ง result ช้าหลัง leaseหมด

4. Trust Boundaries

```

Untrusted player code
  | AST guard (cheap filter, bypassable)
  v
Subprocess process OR Docker container
  | result sentinel + JSON validation
  v
Judge Worker
  | authenticated WORKER_* over TCP (no encryption)
  v
Arena authoritative state
  | display-only snapshots
  v
Unauthenticated UDP feed after attach token
  
```

Container kernel boundary เป็น security boundary ที่แข็งแกร่งที่สุด AST guard ไม่ใช่ boundary และ subprocess บน Windows ให้ isolation แบบ best-effort เท่านั้น

5. Controls

5.1 Protocol and Framing

- TCP frames จำกัดสูงสุด 1 MB; submission จำกัด 256 KB
- **Content-Length** กำหนด boundary และ reader อ่าน body เท่าจำนวน byte
- **Body-SHA256** ตรวจสอบ source/job/result body ก่อนใช้
- Header values จาก user ถูกจำกัด character และ collapse whitespace เพื่อกัน CRLF injection
- Responses echo **Seq**; Events ไม่มี **Seq**; connection มี reader เพียง thread เดียว

- Event outbox จำกัด 256 entries เพื่อกัน slow receiver ใช้ memory ไม่จำกัด

5.2 Authentication and Authorization

- Unknown user และ wrong password ตอบ **401 AUTH_FAILED** เหมือนกัน
- Submission lookup ตรวจสอบ owner; ของผู้อื่นได้ **403 FORBIDDEN**
- Worker ใช้ pre-shared token และ worker id ผูกกับ TCP connection
- Duplicate live worker id ถูกปฏิเสธ
- UDP attach token ใช้ระบุ display subscription เท่านั้น

5.3 Job Reliability

- Worker เป็น client และ long-poll จึงไม่ต้องเปิด inbound port
- Job มี heartbeat lease; สาม missed intervals ทำให้ eject/requeue
- Disconnect requeue job ทันที
- Dispatch เป็น at-least-once แต่ **record_verdict** รับ result แรกเพียงครั้งเดียว
- ไม่มี healthy local/remote judge ทำให้ **SUBMIT** ตอบ **503** ก่อนสร้าง record/cooldown

5.4 AST Guard

Static scan ปฏิเสธ import ที่ไม่อยู่ whitelist, **open, eval, exec, compile, input, globals, locals, vars**, dynamic import และ dunder attribute access

ประโยชน์คือ reject operation ราคาถูกก่อน execute แต่ Python introspection มีหลายทาง bypass จึงเรียก defence-in-depth เท่านั้น **--no-ast-guard** ต้องคงไว้เพื่อพิสูจน์ layer จริง

5.5 Subprocess Backend

- Fresh child, temporary cwd, closed stdin หลัง runner อ่าน job
- Parent wall-clock timeout และ process-tree kill
- stdout/stderr cap 64 KB
- Genuine sentinel result ต้องเป็น sentinel สุดท้าย
- POSIX ใช้ address-space rlimit เมื่อมี
- Windows ใช้ tracemalloc threshold และ optional psutil polling

ข้อจำกัด: child ยังรันด้วย user account เดียวกับ host, อ่าน absolute host path และเปิด network ได้เมื่อ guard ถูก bypass CPU exhaustion จำกัดด้วย wall-clock kill เท่านั้น

5.6 Docker Backend

- **--network none**
- Read-only root filesystem และ explicit read-only **/app** mount
- **/tmp** เป็น small **noexec, nosuid** tmpfs
- Memory/memory-swap cgroup รวม runtime allowance; auxiliary contract ยังตรวจใน runner
- **--pids-limit 8, --cpus 1.0**
- **--cap-drop ALL, no-new-privileges**, UID/GID 65534
- Unique container name, **--rm**, force-remove on parent timeout

Base image provisioning เกิดก่อน submission stopwatch เพื่อไม่ให้ download ถูกนับเป็น algorithm time

5.7 UDP Boundary

UDP ไม่มี ACK, encryption หรือ message authentication นอกเหนือจาก attach token ผู้โจมตีใน LAN อาจ spoof progress ได้ ดังนั้น payload มีเฉพาะ username, match id, clock, passed count และ number of submissions ไม่มี source, password, authoritative verdict หรือ state-changing command

Client ที่ส่ง malformed datagram และ `seq <= last_seen` การ spoof ทำให้ display ผิดชั่วคราวแต่ไม่ เปลี่ยน match outcome ซึ่งยังมาจาก TCP เท่านั้น

6. Empirical Security Test

`python -m experiments.backend_overhead` ปิด AST guard โดยตั้งใจและรัน probe เดียวกันได้ สอง backends บน CPython 3.14.3 / Windows 11:

Probe	Subprocess	Docker
Connect to controlled host listener	Escaped	Blocked by network namespace
Read controlled host-only path	Escaped	Confined; host path unavailable
Spawn 16 overlapping children	Escaped, 16 spawned	Incomplete, 7 spawned before pid cap

ผลนี้แสดงว่า AST guard เป็น first filter แต่ Docker/kernel policy เป็น boundary ที่บังคับจริง Subprocess สามารถ survive probe ได้แต่ไม่ได้ prevent capability นั้น

7. Accepted Risks / Deliberate Limitations

1. TCP ไม่มี TLS: password, source และ Worker token อ่านได้โดยผู้ดัก traffic
2. Password เก็บ plain text ใน memory; ไม่มี persistent database หรือ password hashing
3. Worker token เป็น shared secret ใน command line มองเห็นได้ใน process listing และ replay ได้
4. UDP attach token อาจ spoof/replay และ feed data ปลอมได้
5. AST guard bypassable และห้ามอ้างเป็น sandbox boundary
6. Windows subprocess ไม่มี hard memory cgroup/rlimit
7. Docker bind-mount `/app` read-only ทำให้ container อ่าน project source ได้ แม้อ่าน host ส่วนอื่นไม่ได้
8. Timing inference มี indeterminate classification; run ล่าสุดไม่ class set-based $O(n)$
9. $O(n)$ กับ $O(n \log n)$ แยกได้ไม่สม่ำเสมอที่ input sizes นี้
10. Opcode counting มองไม่เห็น work ใน C builtins เช่น `list.sort()`
11. Worst-case generator ถูกต้องหรือไม่เป็นหน้าที่ problem author
12. In-memory account/match state หายเมื่อ Arena restart

8. Recommended Production Improvements

หากพัฒนาต่อสำหรับ public deployment ควรเพิ่ม TLS/mTLS, password hashing, per-worker credential, persistent database, signed UDP snapshots หรือเปลี่ยน display channel เป็น authenticated QUIC, container image pin by digest, seccomp/AppArmor profile, audit log persistence และ multi-host orchestrator โดยการปรับเหล่านี้อยู่นอก scope ของ Socket Programming coursework

9. Security Conclusion

CDAP ลดผลกระทบจาก untrusted code ด้วย layered controls และระบุข้อจำกัดโดยตรงไปตรงมา ที่สำคัญคือไม่ให้ weak display channel หรือ bypassable static guard กลายเป็น authority: Arena state และ verdict เดินทางผ่าน TCP ส่วน kernel-enforced Docker policy เป็น execution boundary

End of report

Source code, experiment scripts and presentation outline are included in the same repository.